Journal of Systems Architecture 180 (2026) 103954

Contents lists available at ScienceDirect

Journal of Systems Architecture

journal homepage: www.elsevier.com/locate/sysarc

Evaluating real-time and Time-Sensitive Networking performance of containerized applications in Linux systems

Jorge Sasiain *, Jasone Astorga, Mariví Higuero, David Franco, Asier Atutxa, Eduardo Jacob

Dept. of Communications Engineering, University of the Basque Country (UPV/EHU), Faculty of Engineering in Bilbao, Plaza Ingeniero Torres Quevedo, n.1, Bilbao, 48013, Spain

ARTICLE INFO

Keywords:
Real-time
Time-Sensitive Networking
Container virtualization
Kubernetes
Industrial applications

ABSTRACT

The evolution towards flexible and automated smart factories is driven by the capabilities of foundational IT ecosystems such as 5G networks and cloud computing platforms. However, this evolution presents several challenges. Some of them pertain the cloud-native resources of edge clouds, which must be made suitable to comply with the requirements of deterministic communications and applications, such as the emerging virtual Programmable Logic Controllers (vPLCs). Real-time systems and Time-Sensitive Networking (TSN) become essential to address these requirements at the computing and networking levels in the continuum between the factory floor and the edge cloud. This paper exhaustively benchmarks and evaluates real-time and TSN mechanisms and configurations in container-based applications running on Linux systems, measuring several timestamps under different conditions. The obtained results profile the performance of deterministic applications running on general-purpose computing platforms, and provide insights about how to optimally tune these platforms to support the deterministic applications in a mixed-criticality environment.

1. Introduction

The fourth industrial revolution, known as Industry 4.0, encompasses the digital transformation across different vertical industries to achieve unprecedented flexibility and innovation potential. Characterized by principles such as inter-connectivity, intelligence, and integration of the physical and virtual worlds, this ecosystem is enabling the blossoming of novel technologies such as Industrial Internet of Things (IIoT), Cyber-Physical Systems (CPS), Digital Twins, and Adaptive Manufacturing. Furthermore, many enabling technologies of Industry 4.0 are believed to help achieve the goals of the future Industry 5.0 [1].

However, this technology-driven industrial transformation does not materialize without a conscious effort. By definition, the characteristics and requirements of IT (Information Technology) and OT (Operational Technology) communications clash. Best-effort Internet is not suitable for the OT traffic of industrial control processes. This traffic is generally low-bandwidth with predictable periodic characteristics but critically latency-bound, as a late or failed frame delivery can have significant negative repercussions. Hence, to guarantee the necessary determinism, OT networks have traditionally relied on dedicated protocols and hardware that are often proprietary and operate isolated from the rest of the network. Not only does this result in impaired network scalability and contribute to vendor lock-in and higher costs, but also constitutes a fundamental barrier towards Industry 4.0 [2]. Indeed, modern advances in Internet networks and services ranging from software-defined and programmable networking, to end-to-end network automation, and to application cloud-nativization, are key enablers for the realization of Industry 4.0, but their adoption in OT networks is only possible if they converge with IT networks [3].

At the networking level, IEEE Time-Sensitive Networking is the reference technology for this purpose of convergence. TSN standards build upon Ethernet standards to provide bounded low latency and reliability to specific communication streams in spite of their coexistence with other types of traffic, over a converged, shared network infrastructure. A critical challenge lies in unifying these deterministic industrial communications with the aforementioned advances of modern Internet networking. Such is the example of virtual Programmable Logic Controllers (vPLCs), which aim to decouple industrial process and machine control functionality from dedicated hardware in order to provide a cloud-native realization of a traditional PLC [4]. Container-based virtualization, which constitutes a lightweight alternative to the traditional hypervisor-based virtualization, stands as the ideal candidate technology for this purpose [5]. Together with Kubernetes as the de facto container orchestration platform in IT environments, they are

* Corresponding author.
E-mail address: jorge.sasiain@ehu.eus (J. Sasiain).

https://doi.org/10.1016/j.sysarc.2026.103954
Received 23 September 2025; Received in revised form 6 February 2026; Accepted 4 August 2026
Available online 13 August 2026
1383-7621/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY-NC license (http://creativecommons.org/licenses/by-nc/4.0/).

likely to become the building blocks of industrial control applications including the aforementioned vPLCs.

In this framework, the integration of TSN with cloud-based principles for industrial automation presents itself as a novel research trend with very high potential, but also with no lack of remaining challenges to tackle, including many kinds of low-level optimizations of the computing and networking resources of mixed-critically edge clouds where the applications run. In light of these challenges, our contribution is the in-depth benchmarking of granular latencies obtained with different real-time and TSN configurations in a testbed based on Linux servers and a TSN switch. We believe that this evaluation is crucial for two reasons: (1) to assess the impact and influence of different combinations of configurations and tuning options in the determinism of real-time containerized applications, and the degree to which an edge cloud based on Linux servers and off-the-shelf software can satisfy their requirements; and (2) to yield insights that can be leveraged by TSN-aware NFV/SDN orchestrators and control planes which have to perform an intelligent resource allocation and configuration in order to meet time-sensitive application requirements while making the most efficient use of available resources.

The remainder of the paper is organized as follows: In Section 2, existing literature in the matter of providing real-time and TSN capabilities to containers is examined. Section 3 provides the reader with technological context about industrial automation networks, real-time Linux capabilities, and TSN. Section 4 describes in extensive detail the experimental testbed and the hardware and software configurations used to perform the benchmarking tests, to maximize their reproducibility. The main contributions of the paper are presented in Section 5, with the comprehensive performance evaluation of real-time and TSN mechanisms and with the resulting discussions. Finally, Section 6 rounds out the paper with a closing remark.

2. Related work

The interest in adopting container-based virtualization in scenarios that involve deterministic communications is growing as the industry becomes more eager to reap its benefits in terms of flexibility and cost-efficiency, and to partake in new use cases that leverage the capabilities of modern IT networks. Hence, many works concerning real-time container-based virtualization can be highlighted. Among the first works that contemplated containerization to migrate time-sensitive applications from dedicated bare-metal, authors in [6] explore the usage of Docker containers for this purpose, while, in [7], authors perform an experimental evaluation of the performance of containers in real-time operating systems. These articles demonstrate that container-based virtualization can be suitable for real-time workloads, and introduce configuration guidelines to carry it out. Authors in [8] evaluate the performance of real-time scheduling policies to shield real-time containers in an orchestration architecture based on Kubernetes, thus specifically addressing the challenge of supporting real-time workloads that co-exist with best-effort ones. In [9], the author evaluates boot and task scheduling latencies in different guest operating systems and container technologies. Its results show that containers introduce the lowest latencies, yet modern hypervisor-based virtualization technologies can still be competitive.

Several works [10–12], and [13] propose new real-time schedulers that either extend the SCHED_DEADLINE policy of Linux, employ a hierarchy of schedulers, and/or apply a time-triggered approach. These works highlight that the development of novel scheduling algorithms is an ongoing research line that could optimize the performance of real-time containers even further compared to using native Linux scheduling mechanisms.

The networking requirements of real-time applications can be just as crucial as their computing requirements. In this regard, authors in [14] evaluate networking options for real-time applications in Docker containers. To ensure determinism of real-time container applications, a related line of work that has emerged more recently concerns the integration of TSN features into containers. The primarily targeted scenario is industrial automation edge clouds. In [15], authors analyze the provision of TSN awareness to containers and characterize the delay sources and timing inaccuracies involved. Novel approaches for introducing TSN features in Kubernetes networking have also started to emerge, as seen in works [16,17]. These articles introduce Kubernetes-based architectures featuring a dedicated TSN-capable container or module where application containers can transparently offload the functionality to handle their TSN transmissions in the node. Furthermore, as the academia begins to address the demand for supporting real-time applications and TSN functions in cloud environments, it becomes pivotal to develop management and orchestration mechanisms that are aware of this new paradigm. In this context, we can highlight the following contributions: [18–20], and [21].

In addition to the aforementioned state of the art, recent surveys [5, 22] comprehensively review mechanisms and techniques proposed in the literature to achieve real-time container virtualization.

Given the current state of the art, the interest in converging deterministic communications with cloud-based principles is clear, particularly concerning real-time containers in edge clouds. The available literature focuses on evaluating one real-time mechanism or feature, to evaluate a potential development, or to compare performance across operating systems or virtualization stacks. However, we believe that a contribution that jointly evaluates and characterizes in detail all the native Linux mechanisms and configurations to provide real-time and time-aware TSN capabilities to containers in edge cloud environments is missing in the literature.

3. Technological context

This section provides a more detailed introduction to the architecture of industrial automation networks (Section 3.1), the mechanisms to support real-time applications in Linux (Section 3.2), and TSN technology and its support in Linux (Section 3.3).

3.1. Industrial automation networks

As a reference model of industrial automation networks, we consider the views of TSN Profile for Industrial Automation (TSN-IA) [23], 5G-ACIA [24], and 3GPP [25], due to their vision of modern and converged industrial networks. 5G-ACIA and 3GPP conceive a model of industrial automation networks characterized by its composition in three connectivity layers or segments, which are: (1) the factory floor, which incorporates production lines with machines and field devices connected via I/O endpoints; (2) a central edge cloud that can include top-level management functionality of the automation pyramid such as that of SCADA, MES and ERP; and (3) an industrial backbone network that provides connectivity between the factory floor assets and the central edge cloud. PLC functionality may be centralized in the edge cloud, and/or distributed on the factory floor. TSN-IA takes a more bottom-up approach from the perspective of industrial control loops, but shares the overall view, proposing a model based on the concept of CPSs that are conformed across the factory floor and connected to the higher layers.

The communication types or use cases in an industrial automation network can be classified in three groups: *Controller-to-controller* (C2C) communication involves communications between machine control units and/or production line control units located either in a production line of the factory floor or in an on-premises edge cloud. *Controller-to-device* (C2D) communication applies to communications – usually control loops to operate a process or machine– between field devices, e.g. sensors, actuators, drives, and the abovementioned control units. *Device-to-compute* (D2Cmp) communication is generally less critical and comprises interactions between field devices and centralized computing resources usually located in an edge cloud. Thus, in the

context of our work, the optimization of real-time performance is most critical to the C2C and C2D communication types, but finding the adequate configurations is key to efficiently support heterogeneous communication requirements.

3.2. Real-time configuration of Linux systems

There are studies and efforts that describe certain aspects concerning the tuning of real-time Linux systems, as introduced in Section 2. After analyzing the available material including online resources and research papers, we can summarize the main real-time configuration and tuning areas as follows:

- Preemptability: The PREEMPT_RT patch for the Linux kernel is intended to provide it with real-time capabilities. This patch targets a maximization of the amount of points in the kernel code where its execution can be interrupted (i.e. preempted), thus helping reduce the latency upper bound that real-time tasks may experience.
- Hardware configuration: A typical Linux system is not tuned for real-time workloads by default, focusing on resource and power efficiency rather than on determinism. There are several hardware-related options that can be tuned for real-time systems, including disabling Hyper-Threading, and disabling power-saving and frequency-scaling modes in the form of processor power states (C-states), processor performance states (P-states), and other system-specific features like Intel Turbo Boost.
- Scheduling: Real-time scheduling policies, including SCHED_FIFO and SCHED_RR (Round Robin), ensure that tasks will always be scheduled before, and never preempted by, those using a normal policy. Another real-time policy, SCHED_DEADLINE, incorporates awareness of the period and deadline of the task instead of employing a priority-based hierarchy of tasks. A careful assignment of scheduling policies and priorities can be critical to avoid noise in real-time tasks and to support mixed-criticality workloads.
- Isolation: In multi-core systems, a possible configuration consists on splitting the available CPU cores between those that may be allocated kernel tasks with potentially unbounded latencies (timer and other specific interrupts, kernel threads, etc.), and those that are isolated from them. The latter set of CPU cores can be fully dedicated to real-time tasks, isolated from as much non-deterministic kernel noise as possible. In addition, timer interrupt ticks can be shut down from these isolated CPU cores while they only run a single task.
- Affinity: In order to realize a fine-grained assignment of tasks and interrupts to the available CPU cores, task affinity and IRQ affinity can be configured. This is complementary to the isolation configuration, as it can be used to manage how real-time and non-real-time workloads alike make use of CPU resources and thus avoid undesired disturbances.

The preemptable kernel and hardware tuning are, for the most part, binary configurations, but real-time scheduling as well as isolation and affinity configurations involve fine-grained nuances that will be evaluated in detail in Section 5. Note that, for the remainder of the paper, for simplicity the term CPU is used to refer to each logical processor in the operating system rather than to the entire hardware CPU socket.

3.3. Enabling time-sensitive networking on Linux systems

IEEE TSN comprises a toolbox of standards developed by the Time-Sensitive Networking Task Group of the IEEE 802.1 Working Group. These standards provide the following functionalities: time synchronization, bounded low latency, high reliability, and resource management. The primary base standard is IEEE 802.1Q, which is progressively extended with new or revised capabilities developed in amendment standards. The latest published revision of IEEE 802.1Q is IEEE 802.1Q-2022 [26]. Fundamentally, A TSN *stream* is an unidirectional flow of data between *end stations*, i.e. from a *Talker* to one or more *Listeners*. This communication traverses a layer 2 network comprised of TSN-capable *bridges*, and belongs to a *Traffic Class* (TC) characterized by a set of features and requirements of the frames carried in it, such as frame size, periodicity, and maximum latency. TCs are identified by the Priority Code Point (PCP) field that is present in the VLAN tag of an Ethernet frame.

The primary TSN standard to achieve bounded low latency for periodic traffic is IEEE 802.1Qbv [27], which guarantees temporal isolation between time-sensitive streams by computing a network-wide schedule where each TC is assigned a dedicated time slot enforced by egress device ports. 802.1Qbv introduces the concept of a Gate Control List (GCL), in which, for each egress port, the transmission gates for the different TCs open and close over time in repeating intervals. This mechanism is known as the Time Aware Shaper (TAS), and is supported by the capabilities provided by the IEEE 802.1AS standard [28] to synchronize the clocks of all TSN devices. 802.1AS standardizes the generalized Precision Time Protocol (gPTP), a profile of the IEEE 1588 [29] standard (PTP, Precision Time Protocol) optimized for time-sensitive applications.

Over the years, multiple TSN features have been implemented in the Linux ecosystem, enabling the integration of real-time Linux-based systems with TSN. Such is the case, for example, of the Linux PTP project, which implements gPTP, and of several Traffic Control Queuing Disciplines (Qdiscs) that implement traffic shapers [30]. For implementing time-based transmissions, the options are the TAPRIO Qdisc, and the ETF Qdisc. ETF depends on the LaunchTime feature of specific NICs to enable time-triggered transmissions. On the other hand, timestamping support in the NICs is a sufficient requirement to use the TAPRIO Qdisc, which is the implementation of the 802.1Qbv standard.

In Section 5, we evaluate the performance of the TAPRIO Qdisc jointly with the real-time configurations described in Section 3.2.

4. Description of hardware, software, configuration details, and benchmarking tests

To carry out all the measurements presented in this paper, we set up a TSN testbed that is described in this section in detail. Later, details of how the tests are executed are described.

4.1. Hardware and software description

The experimental scenario is composed of two Linux servers connected at 1 Gbps through a TSN-capable switch. One of them is the target of all the configurations evaluated in this paper, as it implements the vPLC. The other server emulates the sensors and actuators that communicate with the vPLC in control loops. Each test is modeled as a control loop that accommodates five simultaneous streams between five sensors, the vPLC, and five actuators. The control loop has a cycle time of one millisecond in which each sensor sends a layer-2 Ethernet frame to the vPLC, which parses it to send a frame to the corresponding actuator. Frames sent by a sensor to the vPLC or by the vPLC to an actuator are spaced out in slots of 200 ms, as shown in Fig. 1. The entire control loop is denoted in the figure as the black and red arrow that goes from a sensor (S1 in the figure) towards an actuator (A1 in the figure), but the scope of this paper is what occurs inside the Linux server with the vPLC, which is the red part of the arrow, i.e. from the timestamp that sensor traffic is input to the controller's NIC to the timestamp that actuator traffic is output from the controller's NIC. Note that the time slots of each stream is not enforced in the indicated egress interfaces during the tests, as the goal is to accurately measure latencies that could exceed the timing allocated to the control loop. The

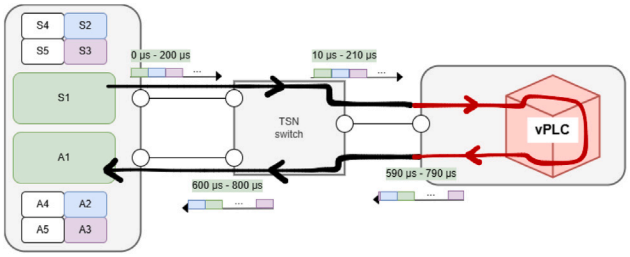


Fig. 1. Experimental scenario and control loop design. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

only exception are the tests that are specifically aimed at evaluating the TAPRIO Qdisc and thus implement a GCL.

The server that implements the vPLC is a Supermicro Super Server with a X11SDV-8C-TP8F motherboard. It is a multicore server with eight Intel® Xeon® D-2146NT CPUs operating at a base frequency of 2.30 GHz. The server's architecture is x86 with a single CPU socket configured as a single NUMA node, and featuring per-core L1 and L2 caches (32 kB L1d + 32 kB L1i and 1 MB L2 per core) and a shared 11 MB L3 cache across the socket. The server contains a total of 32 GB of DDR4 RAM operating at 3200 MT/s. The NIC used for the tests is an Intel® Ethernet Controller I350, using the igb driver at a link speed of 1 Gbps. The controller provides eight Tx/Rx queues with Receive Side Scaling (RSS) and supports full hardware timestamping, exposing a dedicated PTP hardware clock (PHC).

The software suite that enables the experiments executed in this paper includes the following: *LinuxPTP*, to synchronize the clocks of the NICs and operating systems of the testbed to a common time reference; the *TAPRIO* Qdisc, to implement the 802.1Qbv GCL in the network interface of the vPLC; *Isochron*, to generate the cyclic TSN-aware traffic of the streams going from sensor to vPLC and from vPLC to actuator; *stress-ng*, to generate artifical load that stress several subsystems of the vPLC server; *iperf3*, to generate load in the form of best-effort network traffic; and Kubernetes, to virtualize the vPLC under test using container-based virtualization.

4.2. Platform configuration details

The baseline configuration of the hardware and software components that comprise the platform is described as follows:

- **Server**: CPUs #4 through #7 are isolated in the server that runs the benchmarked vPLC through *isolcpus* and *nohz_full* as grub command-line parameters, and with the CONFIG_NO_HZ_FULL and CONFIG_RCU_NOCB_CPU kernel options set. As such, these CPUs are isolated from the default scheduler and have timer tick noise minimized. CPUs #0 through #3 are kept as housekeeping CPUs. Additionally, The servers have Hyper-Threading and P-states disabled in addition to the C-states of all CPUs fixed to C0, meaning the CPUs never halt of sleep. In summary, the grub cmdline in use includes *isolcpus = managed_irq,domain,4–7 nohz_full = 4-7 rcu_nocbs = 4-7 intel_idle.max_cstate = 0 processor.max_cstate = 0 idle = poll*.
- **Network interface**: The network interface used to benchmark the vPLC streams uses VLAN subinterfaces, specifically one for each of the five streams. They use an *egress-qos-map* configuration of *0:0 1:1 2:2 3:3 4:4 5:5 6:6 7:7*, meaning that each socket priority (SO_PRIORITY) value is mapped to the same VLAN PCP value. The PCP value is used by the TSN switch to apply the right treatment to each vPLC stream. The network interface supports TCP Segmentation Offload (TSO), Generic Segmentation Offload (GSO), and Generic Receive Offload (GRO), and has the three capabilities enabled by default. The *irqbalance* daemon is enabled by default, which distributes hardware interrupt load from the eight Tx/Rx queues of the NIC across the housekeeping CPUs. The NICs ring buffer's size is set to 256 Socket Buffers (skb) for both Tx and Rx.
- **LinuxPTP**: The server that runs the vPLC and the server that emulates the sensors and actuators are synchronized via gPTP. In each server, the *ptp4l* daemon synchronizes the PHC of the NIC, and the *phc2sys* daemon synchronizes the system clock (CLOCK_REALTIME) to the PHC. Both daemons run with a default scheduling priority and in housekeeping CPUs. The grandmaster clock that the servers' PHCs syncrhonize to is provided by the TSN switch. The gPTP messages are sent with a PCP of 0.
- **Kubernetes**: The Kubernetes cluster provides the environment where the vPLC runs. Each stream that originates or terminates on the vPLC involves a individual container that is responsible for running the Isochron sender or receiver process. The containers are deployed with *hostNetwork: true*, allowing them to use the host's network stack directly. This eliminates CNI and virtual networking overhead, which is outside the scope of this paper. *containerd* is used as the container runtime. Configuration details concerning how the containers are built and instantiated are provided in Section 4.3.

4.3. Measurement tests configuration details

This subsection describes how each tool and software component used to carry out the measurements in Section 5 is used and configured.

As introduced in Section 4.1, each test consists on periodic streams being sent to or from the vPLC, which are measured across several timestamps. The former measure the receive path (representing messages from sensor devices) and the latter the transmit path (representing message to actuator devices). Each test involves 1.000 consecutive iterations of each of the five streams.

Isochron is the software tool used to send the streams and benchmark the transmit and receive path latencies. With Isochron, cyclic layer-2 frames are sent and received over a specific VLAN device of the server. Each stream uses its own SO_PRIORITY and VLAN PCP, which are aligned with the traffic classes configured in the TAPRIO Qdisc in the tests in which TAPRIO is in use. Stream #1 uses SO_PRIORITY 1 and PCP 1, stream #2 uses SO_PRIORITY 2 and PCP 2, and so on. The cycle time is configured as 1 millisecond and the shift time of the streams are each 0, 200, 400, 600, and 800 ms. Isochron is indicated to wake up for sending a frame at the exact moment where that frame is supposed to be sent according to this schedule, with guarantees that wake up latency is timestamped accurately in the tests measuring the transmit path of the vPLC. Lastly, Isochron is indicated to start only once it detects that the gPTP synchronization accuracy between the benchmarked vPLC server and the other server emulating the field devices is no worse than 200 ns. The Isochron sender and receiver processes are single-threaded, and thus can be pinned to a single specific CPU.

Isochron measures the following timestamps:

- Scheduling latency of the Isochron application, measured as the time it takes to wake up since its scheduled time (wake_up in Section 5 figures).
- The SOF_TIMESTAMPING_TX_SCHED timestamp, which is taken just before the packet is handed to the scheduling of the Qdisc layer (tx_qdisc in Section 5 figures). This timestamp captures protocol processing latency.
- The SOF_TIMESTAMPING_TX_SOFTWARE timestamp, which is taken in the network stack just before the packet is passed from the device driver to the NIC (tx_sw in Section 5 figures). The difference between this timestamp and SOF_TIMESTAMPING_TX_SCHED captures queuing and processing latency at the Qdisc layer.

- The SOF_TIMESTAMPING_TX_HARDWARE timestamp, which is taken just when the packet leaves the NIC for transmission (tx_hw in Section 5 figures). The difference between this timestamp and SOF_TIMESTAMPING_TX_SOFTWARE captures the latency at the NIC (reading the packet bits and putting them on the wire).
- The SOF_TIMESTAMPING_RX_HARDWARE timestamp, which is taken by the NIC when the packet arrives, just before raising the hardware interrupt (rx_hw in Section 5 figures).
- The SOF_TIMESTAMPING_RX_SOFTWARE timestamp, which is taken just when the packet is passed to the network stack of the kernel (rx_sw in Section 5 figures). The difference between this timestamp and the SOF_TIMESTAMPING_RX_HARDWARE timestamp captures the latency of servicing the interrupt, which passes the packet up to the kernel protocol stack.

To further benchmark the receive path up to the userspace, an additional C program is used to record an additional timestamp that correspond to the instant when the payload of the packet has been fully read into userspace memory. It listens on the network interface for frames with the expected EtherType. It creates an AF_PACKET socket and enters a *recvfrom()* loop to receive the packets from the socket's receive queue. A timestamp is generated after a packet is detected and its entire payload is read (rx_read in Section 5 figures). This timestamp captures kernel protocol stack latency, as well as userspace latency for reading the bits of the packet's payload.

Some tests employ additional software tools to introduce artificial load that emulates realistic processing or networking workloads. As mentioned before, these tools are stress-ng and iperf3. The stress-ng configuration is *stress-ng –cpu $(nproc) –cpu-method all –vm $(nproc) –vm-bytes 45% –vm-keep –sock $(($(nproc)/2)) –iomix $(($(nproc)/4)) –fork $(($(nproc)/2)) –schedmix $(($(nproc)/2))*, aimed at representing a realistic scenario of a data center server running virtualized workloads. This yields the following stressors: *cpu (4) vm (4) sock (2) iomix (1) fork (2) schedmix (2)*. Only some measurements at the end of Section 5.2 use a different stress-ng configuration, which is described there. Regarding the use of iperf3, each execution triggers four parallel streams between both servers in the same direction as the path being benchmarked. The bandwidth and transport layer protocol used is test-specific and indicated in each test. Each stream uses the same amount of bandwidth. For example, *-P 4 -b 250M -u -R* are the parameters for 100% of the 1Gbps bandwidth of the NIC using UDP and in the direction from sensor to vPLC. Both stress-ng and iperf3 are scheduled over housekeeping CPUs and with default SCHED_OTHER priorities. Iperf3 traffic gets best-effort treatment with SO_PRIORITY 0 and no VLAN tag. Tests that are executed together with neither of stress-ng or iperf3 are later referred to as tests with the vPLC server in idle state, which only involve background processes, the Kubernetes control plane, and the gPTP daemons.

Concerning the configuration of real-time characteristics of containerized applications deployed with Kubernetes, Kubernetes does not provide a native way to configure real-time scheduling policies and priorities in the application running inside containers. Hence, this configuration has to be imprinted into the container's executable. Container images for the Isochron sender, Isochron receiver, and the userpace listener are constructed using a *Containerfile* and building it. In such file, the ENTRYPOINT or CMD instruction is used to specify the command that starts the main process when the container begins to run. The assignment of a real-time priority to the main process inside the container, and subsequently to any thread created by it, can be configured in either instruction using the *chrt* command. An example of assigning a SCHED_FIFO policy with a priority of 40 is as follows: *CMD ["chrt", "-f", "40", "./app"]*. Alternatively, the *command* field of the pod in the Kubernetes manifest file can override the image's default command without requiring the image being rebuilt. In either case, in order for the container to acquire the privilege to configure process priorities and scheduling policies, the SYS_NICE capability has to be specified in the *securityContext* field of the container's configuration of the manifest file.

In regards to CPU isolation. Kubernetes provides the CPU Manager feature, which includes a static CPU assignment policy for containers. It allows to provide access to an integer number of exclusive cores to a pod, provided that a quota configuration that places it in the Guaranteed QoS class is provided in the manifest file. However, the CPU Manager does not allow to assign specific CPU numbers to the pods, which makes it unsuitable to provide a time-based allocation of CPUs aligned with the TSN time schedule for the TSN streams of each application, or to synchronize CPU assignment with NIC interrupt affinity configurations. Hence, we use the same approach followed to configure process scheduling, which is applying CPU affinity to the ENTRYPOINT or CMD instruction of the Containerfile or the *command* field in Kubernetes, using the *taskset* command. The following example includes both real-time scheduling and CPU affinity: *CMD ["taskset", "-c", "2", "chrt", "-f", "40", "./app"]*.

Listing 1 shows a reference skeleton of the deployment manifest file used to generate the containers. It describes an application with real time scheduling and a pinned CPU (lines 20–21), the SYS_NICE capability (lines 23–25), host networking (line 26), and node selection (line 27). In the tests, the receiver processes use Deployments, whereas the Isochron senders use Jobs that begin simultaneously and complete after the 1000 cyclic frames have been sent.

Listing 1: Example deployment manifest file

```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: vplc1
5    labels:
6      app: vplc1
7  spec:
8    replicas: 1
9    selector:
10     matchLabels:
11       app: vplc1
12   template:
13     metadata:
14       labels:
15         app: vplc1
16     spec:
17       containers:
18       - name: vplc1-container
19         image: vplc1
20         command: ["taskset", "-c", "2", "chrt",
21                   "-f", "40", "./app"]
22         securityContext:
23           capabilities:
24             add:
25             - SYS_NICE
26       hostNetwork: true
27       nodeName: k8s-node-a
```

As stated before, the TAPRIO Qdisc is used to implement the TAS of the 802.1Qbv standard in some of the tests where transmit path performance is evaluated. Configuration of the TAPRIO Qdisc involves the definition of a mapping between Linux SO_PRIORITY values, TCs, and the NIC's Tx queues. Next, a GCL is defined, which includes entries with the time slot duration and the gate operations (gate open or gate closed) for each TC. The cycle time is implicitly the sum of all time slot durations. Additionally, the TAPRIO Qdisc configuration is complemented by the configuration of the VLAN device of the interface, which associates each of the eight VLAN PCP values with a Linux SO_PRIORITY value. For the tests performed in this paper, each of the five streams is assigned a TC, and each of these TCs has one of the NIC's eight available transmit queues allocated to it. The scheduling entries are configured as indicated in Section 5 according to the individual test.

Table 1
Summary of the measurements carried out in Section 5.

Section	Figures covered	Description
Section 5.1	Figs. 2–4	Assessment of baseline receive path performance under different kinds of system load.
Section 5.1	Figs. 5–10	Evaluation of the impact of real-time scheduling and exclusive CPU affinity on receive path performance.
Section 5.1	Figs. 11–12	Evaluation of the impact of IRQ pinning and other mechanisms on receive path performance
Section 5.2	Figs. 13–16	Assessment of baseline transmit path performance under different kinds of system load.
Section 5.2	Figs. 17–19	Evaluation of the impact of real-time scheduling and exclusive CPU affinity on transmit path performance.
Section 5.2	Figs. 20–22	Evaluation of the impact of time-aware shaper using the TAPRIO Qdisc on transmit path performance.

5. Benchmarking results and discussions

5.1. Overview of the executed benchmarking tests

The aim of this section is to assess the impact on latency and jitter of different real-time configurations across the entire receive and transmit path of the control loop in the vPLC. As such, several timestamps that capture the latencies at the NIC, kernel, and userspace levels are evaluated. In summary, the impact of configurations and mechanisms like frame length, real-time scheduling, exclusive CPU affinity, IRQ pinning, and Time-Aware Shaper are evaluated, both when the server is in idle state or under stress.

Table 1 provides a A classification of the measurements carried out in Section 5 in chronological order. The "description" column indicates the aim of each group of tests, considering that the component under evaluation is always the vPLC, and, in particular, the control-loop streams send by or received by it.

The results of the tests performed in this section are represented using boxplot graphs. In these figures, the boxes represent the samples below the 99th percentile, with a solid line indicating the median of all samples and a dashed line indicating the mean of all samples. Dots represent the worst-case latency samples above the 99th percentile. In addition, the white area of the graph utilizes a linear scale in the Y axis, whereas the gray area utilizes a logarithmic scale more suitable to fit the worst performing tests.

5.2. Benchmarking of receive path latencies from sensors to vPLC

The first batch of tests are aimed at determining a baseline performance of the receive path when the server is in idle state, including the impact of packet size. Fig. 2 shows that the latency incurred in the receive path since the NIC receives the frame from the wire until the it enters the kernel averages around 5 ms, with worst-case samples going up to around 30 ms. The path through the kernel protocol stack takes longer, with worst-case samples reaching 100 ms. Fig. 3 depicts an aggregate of the five streams side by side with larger frame lengths. The results indicate that frame length has a significant impact on the RX_SOFTWARE latency, suggesting a linear increase of nearly 1 µs per 100 additional bytes of frame length. The impact of frame length on the latency from kernel to userspace is smaller and also linear in regards to the average value, as suggested by the RX_READ boxplots. As stated in Section 4, subsequent measurements will employ a frame length of 64 bytes for the vPLC streams unless otherwise specified.

Figs. 4 and 5 illustrate the measured latencies when the system is under the two types of artificial load described in Section 4. Fig. 4 corresponds to the non-protected Isochron vPLC streams having to contend for resources with stress-ng stressors. While the impact on RX_SOFTWARE latency is minimal, the remainder of the receive path is substantially affected, with worst case RX_READ samples approaching 4 ms. Meanwhile, Fig. 5 reflects the impact of iperf3 traffic streams being sent to the vPLC at the same time as the control loop streams. Each individual boxplot in the figure shows an aggregate of the five vPLC streams (stream #1 through #5) of the measurement. The eight boxplots displayed in the figure per timestamp correspond to different iperf3 configurations as denoted by their labels: *T* and U mean TCP and UDP traffic respectively, and the number indicates the bandwidth

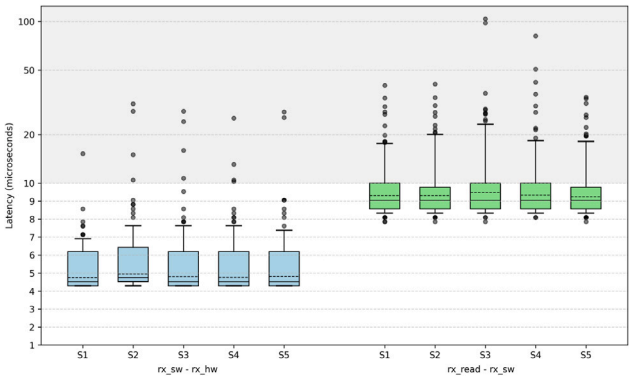


Fig. 2. Receive path latencies per stream with server in idle state.

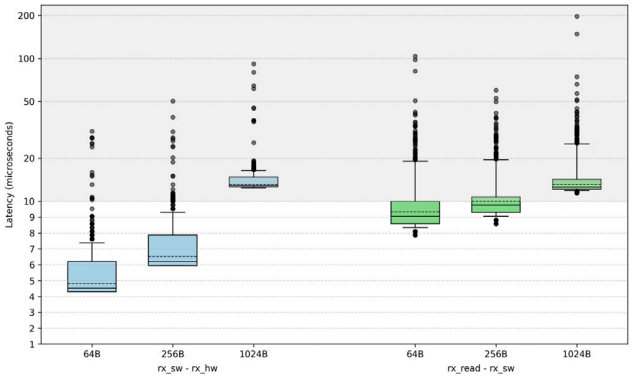


Fig. 3. Receive path latencies by frame length with server in idle state.

taken up by the iperf3 streams relative to the maximum speed of the NIC. As seen in the figure, the control loop streams see their performance degraded progressively as the bandwidth occupied by the iperf3 stream increases. The average RX_SOFTWARE latencies are minimally affected, but RX_READ latencies present volatile results of more than 1 millisecond in the worst cases.

After these preliminary tests, the next batch of tests are aimed at benchmarking the impact of real-time scheduling and CPU affinity in the worst case and average latencies.

Firstly, on the one hand, to assess the impact of the different priority levels of both RT and non-RT policies, the tests depicted in Figs. 2, 4, and 5 were repeated with three additional scheduling configurations in all five of the actuator processes: the default SCHED_OTHER priority used in the aforementioned figures, a SCHED_OTHER priority with -20 nice value, a RT SCHED_FIFO priority of 40, and a RT SCHED_FIFO priority of 80. The primary implication of these two RT priority levels is that one of them is located above the priority value associated to the default threaded interrupt handler under the PREEMPT_RT patch, which is 50, and the other priority value is located below it. Still, no difference was found between either the two SCHED_OTHER priority levels or the two SCHED_FIFO priority levels. Thus, in the remainder of this subsection, for brevity, subsequent non-RT measurements are

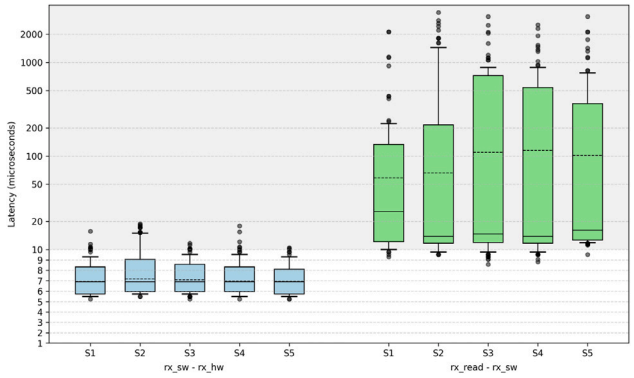


Fig. 4. Receive path latencies with server under stress-ng load.

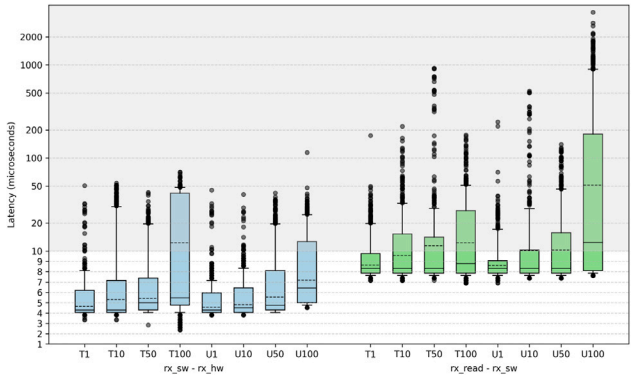


Fig. 5. Receive path latencies with server under iperf3 load.

executed with the default SCHED_OTHER priority and subsequent RT measurements are executed with a SCHED_FIFO priority of 80.

On the other hand, to assess CPU contention between the five vPLC streams, which do not overlap in time as pointed out by Fig. 1, the receiver processes of stream #1 were pinned to one isolated CPU, and those of streams #2 through #5 all to a second isolated CPU. As expected, the CPU sharing of streams #2 through #5 had no effect in the measured latencies compared to stream #1. Following this finding, to simplify the subsequent figures, they display the aggregate of the five streams for each given metric, with all processes being pinned to the same isolated CPU in the measurements that CPU pinning is indicated to be applied.

The results of protecting the receiver processes of the vPLC streams through real-time scheduling and CPU affinity are shown in Fig. 6 (64-byte frames) and in Fig. 7 (1024-byte frames). In these figures, the measured latencies are compared side-by-side with four configurations: (BE) the unprotected configuration of Fig. 2; (RT) assigning the processes a SCHED_FIFO priority of 80; (CPU) pinning the processes to an isolated CPU; and (RT+CPU) combining the previous two. With the system not being under stress, neither of the two types of protection is particularly significant at improving the latencies achieved with the best-effort configuration, although real-time scheduling slightly reduces the average RX_READ latency, and worst-case samples are also minimized overall.

In the subsequent measurements reflected in Fig. 8 (64-byte frames) and Fig. 9 (1024-byte frames), the same real-time scheduling and CPU pinning configurations are applied again, but adding the stress-ng artificial load described in Section 4. These figures show that the use of real-time scheduling affects the measured RX_READ latencies positively, reducing the average values significantly. However, the worst-case samples that approach 1 millisecond are still unacceptable for the control loop requirements. Due to the uncertainty of this test,

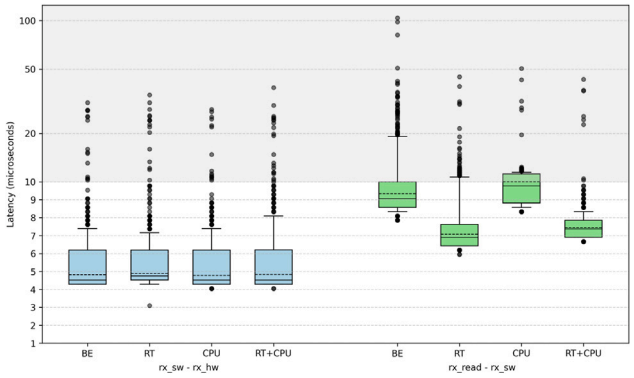


Fig. 6. Receive path latencies with server in idle state using real-time scheduling and CPU isolation.

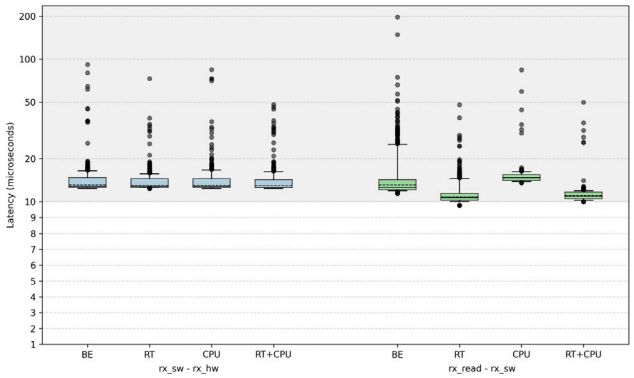


Fig. 7. Receive path latencies with server in idle state using real-time scheduling and CPU isolation (1024-byte frames).

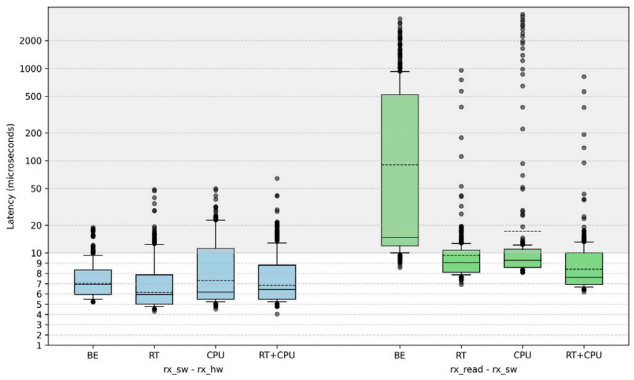


Fig. 8. Receive path latencies with server under stress-ng load using real-time scheduling and CPU isolation.

it was run a total of five times, yielding a small amount of worst-case samples around 500–1000 ms in most executions.

Fig. 10 displays the results of the measurements with real-time scheduling and CPU pinning configurations alongside the iperf3 artificial load described in Section 4 The frame length used in the vPLC streams is 1024 bytes in order to evaluate the case that caused slightly worse latencies in previous measurements. Unlike the case of Fig. 9, the results in Fig. 10 reflect that the combination of real-time scheduling and affinity to an exclusive CPU is considerably successful at shielding the receive path of the vPLC stream against this type of iperf3 load. Although the latencies are worse than without the background iperf3 traffic (case RT+CPU of Fig. 6), the average latencies are still low and

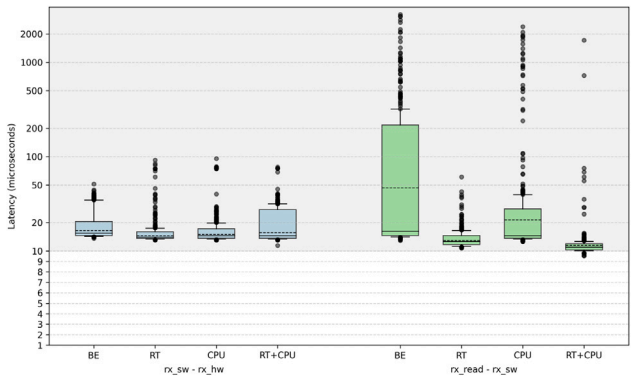


Fig. 9. Receive path latencies with server under stress-ng load using real-time scheduling and CPU isolation (1024-byte frames).

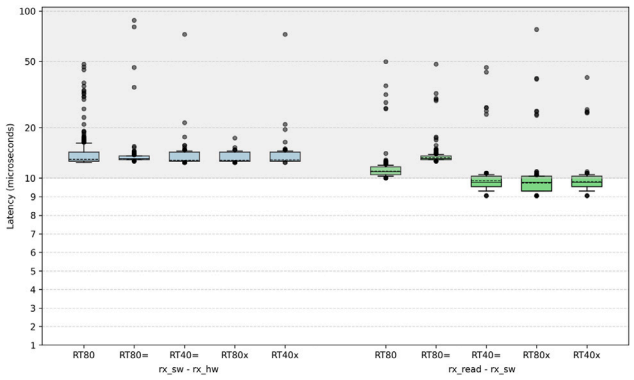


Fig. 11. Receive path latencies with server in idle state using real-time scheduling, CPU isolation, and IRQ pinning (1024-byte frames).

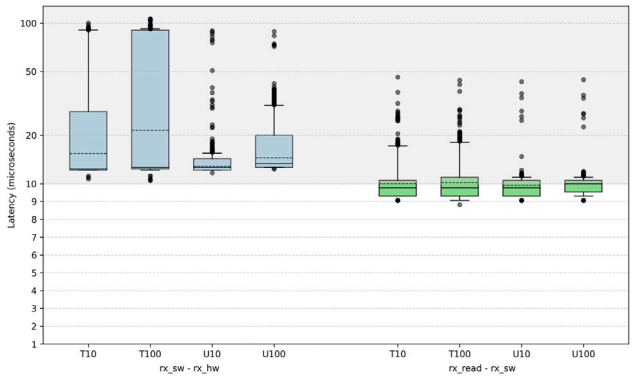


Fig. 10. Receive path latencies with server under iperf3 load using real-time scheduling and CPU isolation (1024-byte frames).

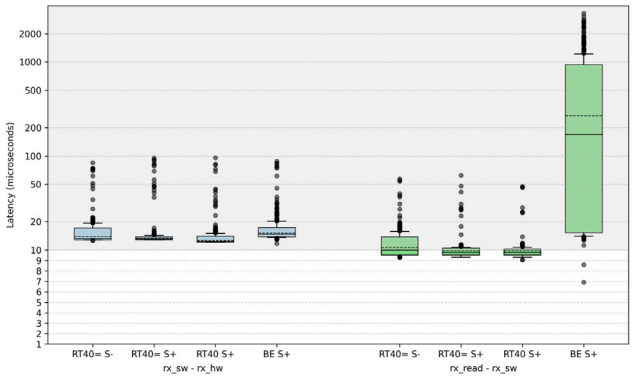


Fig. 12. Receive path latencies with server under different stress-ng load configurations using real-time scheduling and CPU isolation (1024-byte frames).

worst-case samples do not exceed 150 ms across the entire measured receive path. This is a significant improvement over the case with no real-time scheduling or CPU pinning (Fig. 5). Like the previous test, this test was run a total of five times to ensure the validity of the results.

Just like specific CPUs can be pinned to specific processes through CPU affinity configurations, hardware interrupts can also be bound to specific CPUs. This is done by setting the CPU affinity mask at */proc/irq/IRQ#/smp_affinity* for each *IRQ#* associated to the network interface and its queues. We can thus assess the impact of isolating the IRQs of the network interface queues to specific CPUs, as well as balancing the scheduling priority of the measurement processes either above or below the priority assigned to the IRQs. The results of this test, using 1024-byte frames, are show in Fig. 11. In the x-axis labels of the figure 40 and 80 correspond to the priority used for the scheduling of both the Isochron receiver process and the userspace receiver process, whereas the = and x icons mean whether the isolated CPU pinned to the IRQs and to the measurement processes is, respectively, the same or a different one. Neither configuration shows a meaningful improvement relative to not isolating IRQs (which in the figure are the leftmost boxplots labeled as RT80). The worst average latencies are obtained in the configuration in which the userspace receiver contends with network interrupts in the same CPU, when the application has a higher scheduling priority than the interrupt handler.

The same IRQ pinning test was conducted alongside the two kinds of artificial load: stress-ng and iperf3. The results obtained are respectively very similar to the ones in the "RT+CPU" boxplot of Fig. 9 and in Fig. 10, which depict the results of the equivalent measurements without IRQ pinning. Thus, pinning the hardware interrupts of the network interface to an isolated CPU was not found to provide a meaningful benefit as worst-case samples of hundreds of milliseconds still occurred sporadically when in contention with stress-ng.

In light of these results, we next inspected how different stress-ng configurations affected the real-time streams. Specifically, the impact of the different stressors was analyzed individually on the measured latencies, and less aggressive configurations were evaluated. This study led to the determination that the worst-case spikes were caused by the fork and schedmix stressors. Furthermore, among the selected stressors for the stress-ng configuration used thus far, these two are considered to be the least representative of the normal workloads in the data center edge-computing scenario intended to emulate. Fig. 12 presents results obtained with stress-ng configurations that do not use these two stressors. The new labels S+ and S- in the figure indicate the amount of the other stressors used. S+ corresponds to the one used so far, whereas S- corresponds to half that amount of each (2 CPU, 2 VM, 1 sock, and 1 iomix). The results illustrate that regardless of the amount of these stressors and of whether hardware interrupts of the NIC are pinned to an isolated CPU or not, worst case samples are contained within less than 200 ms across the entire Rx path. This highlights that assessing the environment that the real-time control loop contends with is highly important to determine the viability of protecting said control loop.

Following these results, the tuning of other parameters that could impact performance of the receive path was investigated.

On the one hand, the impact of NAPI and software interrupts (softirqs) was assessed. The hardware interrupt handler for receiving packets performs minimal work itself, instead scheduling the NAPI poll function. NAPI is a Linux mechanism that defers packet processing into softirq context via the NET_RX softirq vector, entering into polling mode for incoming packets. If the amount of softirqs reaches a threshold, the *ksoftirqd* kernel thread is wake up to process the

remaining softirqs. As a test, the ksoftirqd thread that services the software interrupts in the same isolated CPU that the hardware interrupts are pinned to was upgraded to use a real-time scheduling policy of SCHED_FIFO. Since this CPU only services the hardware IRQs of the network interface, only the NET_RX software IRQs of this network interface should be affected by this change. However, after executing the tests, it was observed that the ksoftirqd did not consume any CPU time, and, this change did not improve latency results.

In addition to the above, tests where the value of kernel parameter *net.core.netdev_budget_usecs* was lowered to 200 from the default of 2.000 were performed. This parameter dictates the maximum number of ms of each NAPI polling cycle. Firstly, we observed in */proc/net/softnet_stat* that with the default netdev_budget the polling cycle was not running out of time, as the third column of the output, which indicates it per CPU, did not increase in any CPU with either iperf3 load or protected vPLC traffic. Still, after lowering netdev_budget_usecs to 200, no change was observed on vPLC stream performance across either of the timestamps measured and with either type of artificial load.

On the other hand, disabling GRO was measured. GRO is a software implementation of Large Receive Offload (LRO), which the evaluated NIC does not support. GRO coalesces incoming packets inside the NAPI pool before passing them up the network stack, to reduce packet header processing in the upper layers. The goal of this test is to determine whether GRO may introduce undesired latency due to aggregating packets. Still, the status of GRO was not found to incur any effect on latency under any conditions (idle state or either type of background load). This is a reasonable outcome as GRO only merges packets from the same pool cycle, so any latency that GRO would introduce would be tied to the NAPI polling cycle duration.

Due to the lack of meaningful differences in the results of the NAPI and GRO evaluations relative to previous results, they are not graphically presented for brevity.

5.3. Benchmarking of transmit path latencies from vPLC to actuators

As the previous measurements have focused on receive path performance, in subsequent tests we benchmark the transmit path, which corresponds to the vPLC sending messages to actuator devices that could be in a machine or robot. Like in the previous subsection, the first batch of tests are aimed at profiling the performance when the server is in idle state and with no mechanisms in place to protect the control loop communication. Fig. 13 reflects that the real-time streams sent by the vPLC are subject to significant jitter, with the process wake up latency contributing with the highest worst-case samples, followed by the TX_SOFTWARE latency. This baseline test is repeated in Fig. 14, increasing the frame length of the streams from 64 bytes to 1024 bytes. The results indicate that the impact of packet size is inconsequential to jitter and worst-case latencies while being minimal to average latencies, which see an increase of less than 2 us across the entire Tx path after the frame length increase. Thus, in the remainder of this subsection, for brevity, subsequent tests are run with a frame length of 64 bytes, which is more representative of cyclic industrial control loop traffic.

Figs. 15 and 16 illustrate the measured latencies when the system is under the two types of artificial load using the configuration described in Section 4. Hence, Fig. 15 corresponds to the non-protected Isochron vPLC streams having to contend for resources with stress-ng stressors. The worst-case spikes are highly noticeable in the wake up and TX_SOFTWARE latencies, with the former going up to 2 ms and the latter to nearly 1 ms. On the other hand, this type of system load does not impact TX_SCHED or TX_HARDWARE latencies. Meanwhile, Fig. 16 reflects contention with the iperf3 traffic streams. Like in the equivalent measurement of the previous subsection, each individual boxplot in the figure shows an aggregate of the five vPLC streams. The eight boxplots displayed in the figure per timestamp correspond to different iperf3 configurations as denoted by their labels. The wake up latency is omitted in this figure as the goal is assessing the impact on

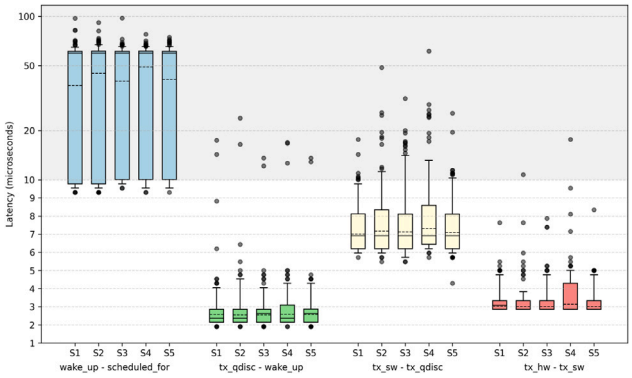


Fig. 13. Transmit path latencies per stream with server in idle state.

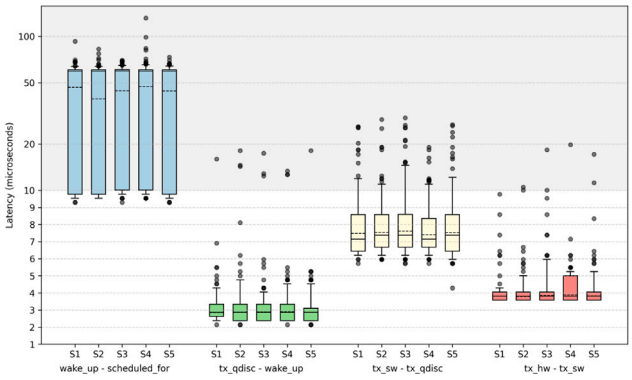


Fig. 14. Transmit path latencies with server in idle state (1024-byte frames).

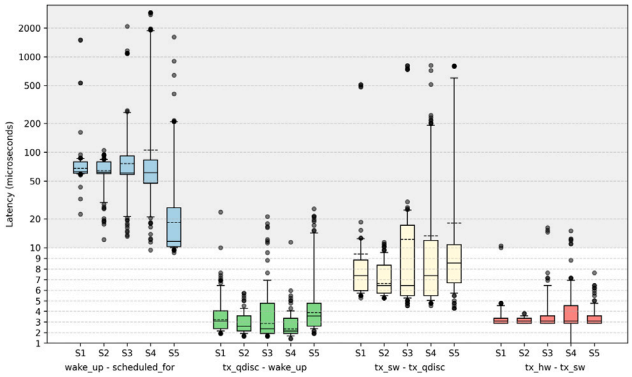


Fig. 15. Transmit path latencies with server under stress-ng load.

transmit path latencies. The figure evidences that the TX_SOFTWARE and TX_HARDWARE latencies of the vPLC streams are significantly affected even when iperf3 takes up just 1% of the bandwidth. Background UDP load is overall less detrimental than TCP.

After these preliminary tests, the next batch of tests are aimed at benchmarking the impact of real-time scheduling and CPU affinity in the worst case and average latencies. In order to assess the impact of the different priority levels of both RT and non-RT policies, the tests depicted in Figs. 13, 15, and 16 were repeated with three different scheduling configurations in all five of the actuator processes Like in the previous subsection, these are: a default SCHED_OTHER priority, a SCHED_OTHER priority with −20 nice value, a RT SCHED_FIFO priority of 40, and a RT SCHED_FIFO priority of 80. Again, no difference was found between either the two SCHED_OTHER priority levels or the two SCHED_FIFO priority levels. Thus, subsequent non-RT measurements

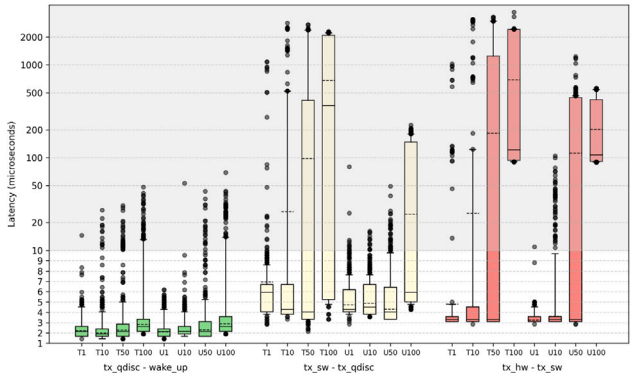


Fig. 16. Transmit path latencies with server under iperf3 load.

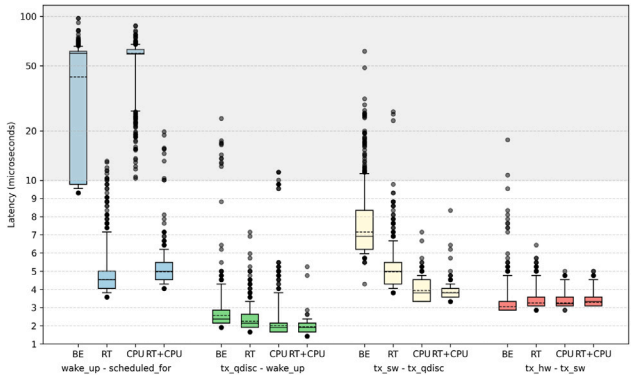


Fig. 17. Transmit path latencies with server in idle state using real-time scheduling and CPU isolation.

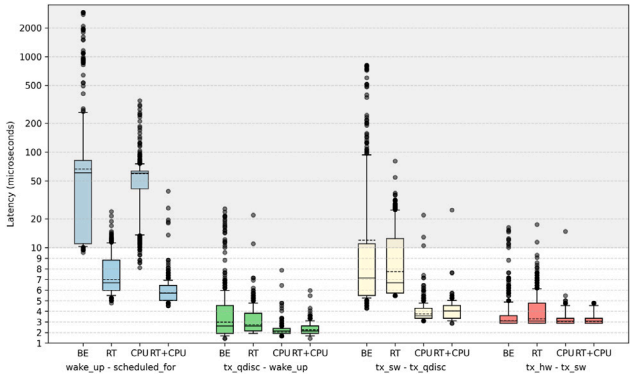


Fig. 18. Transmit path latencies with server under stress-ng load using real-time scheduling and CPU isolation.

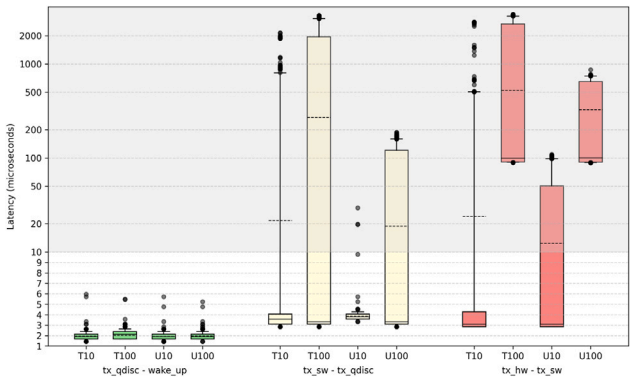


Fig. 19. Transmit path latencies with server under iperf3 load using real-time scheduling and CPU isolation.

are executed with the default SCHED_OTHER priority and subsequent RT measurements are executed with a SCHED_FIFO priority of 80. Likewise, to assess CPU contention between the five vPLC streams, stream #1 was pinned to one isolated CPU, and streams #2 through #5 all to a second isolated CPU, which had no impact on the measured latencies. Hence, subsequent figures display the aggregate of each of the five streams for any given metric, with all streams being pinned to the same isolated CPU in the measurements that CPU pinning is indicated to be applied.

The results of protecting the vPLC streams through real-time scheduling and CPU affinity are shown in Fig. 17. In this figure, the measured latencies are compared side-by-side with three configurations: (RT) assigning the vPLC streams an SCHED_FIFO priority of 80; (CPU) pinning the vPLC streams to an isolated CPU; and (RT+CPU) combining the previous two. The results show that CPU isolation is effective at reducing worst-case latency along the transmit path, especially at the TX_SOFTWARE timestamp, whereas using a real-time scheduling priority significantly reduces the wake up latency of the process, reducing worst-case samples from 100 ms to 20 ms.

In the subsequent measurements reflected in Figs. 18 and 19, the same configurations are applied again but adding the two types of artificial load described in Section 4. Fig. 18 shows that the combination of CPU affinity and real-time scheduling is very robust at protecting the vPLC streams against the stress-ng load. Compared to the results without artificial load of Fig. 17, wake up latency degrades if the scheduling is not real-time, whereas the other three timestamps that represent the transmit path along the network stack yield worse latencies if CPU isolation is not used. However, when the vPLC streams use both real-time scheduling and affinity to an isolated CPU, average latencies remain virtually identical, and worst-case samples remain bounded at tens of ms, with the end-to-end Tx path never going above 100 ms. Overall, this is over an order of magnitude better than the results under stress-ng load without neither real-time scheduling or CPU isolation, which are the boxplots labeled as BE. On the other hand, contention with the iperf3 load (Fig. 19) does not yield the same results, manifesting that CPU affinity and real-time scheduling alone are not enough to shield the vPLC streams against contending network traffic.

In order to address protection mechanisms against contending traffic, our next aim is to evaluate time-based TSN. For this purpose, we will initially benchmark the impact of using the Time-Aware Shaper (TAS), which in Linux is implemented in software through the TAPRIO Qdisc. Before evaluating how successful TAPRIO is at shielding the real-time vPLC streams with a GCL, the impact of simply attaching the Qdisc will be characterized. Fig. 20 compares the latencies obtained when the TAPRIO Qdisc is used with a GCL with 5 gate operations and with a GCL with 25 gate operations. In this test, all gates remain open as the goal is to simply measure whether TAPRIO introduces any significant delay compared to the case where it is not used.

The contribution of TAPRIO processing to transmit path latency manifests in the TX_SOFTWARE timestamp of Fig. 20. The average latencies increase insignificantly, and even though the amount of latency spikes seem to increase, worst-case samples cap at around 30 ms, which is an affordable amount of time in the control loop. The identical results of TAPRIO with 5 gate operations and with 25 gate operations reveals that the number of gate operations does not impact the latency incurred by the Qdisc, at least for a realistic number of GCL gate operations.

Fig. 21 incorporates a TAPRIO GCL and shows latencies across the transmit path when alongside different kinds of best-effort iperf3 traffic. To perform this test, initially, two of the five streams (stream #2 and stream #4) were given an exclusive gate, while the duration of the other three (a 200-microsecond slot plus a 400-microsecond slot across

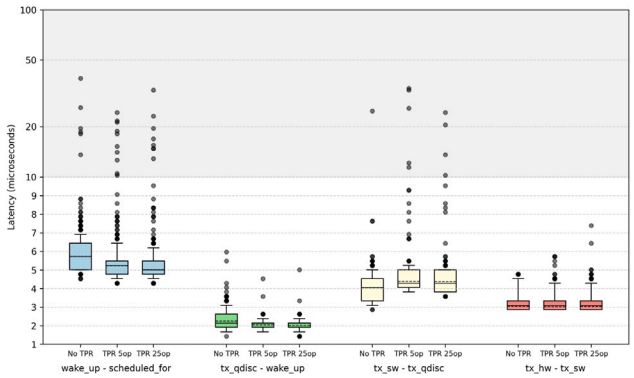


Fig. 20. Transmit path latencies with server in idle state using real-time scheduling, CPU isolation, and the TAPRIO Qdisc.

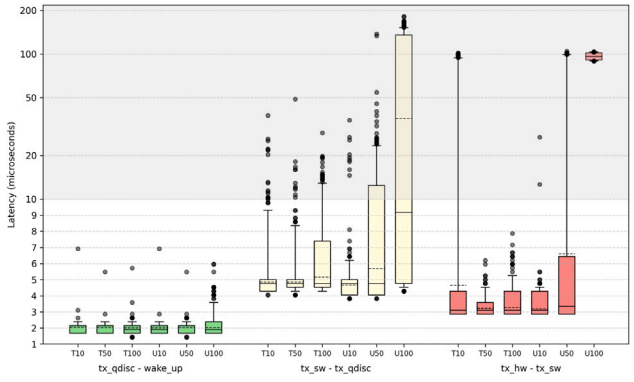


Fig. 21. Transmit path latencies with server under iperf3 load using real-time scheduling, CPU isolation, and the TAPRIO Qdisc.

each 1 ms cycle time) was instead assigned to the best-effort traffic. The first thing that we realized is that the tests with TCP nearing the maximum available bandwidth did not complete. Rather, both iperf3 caused the network interface to go down. nuttcp was also tested with similar results. After examining different TAPRIO configurations, we identified that the smallest best effort gate duration necessary for the TCP test to complete was approximately 300 ms. Thus, we changed the configuration to assign a exclusive gate to streams #1 and #2, ensuring that best-effort traffic had a consecutive slot of 600 ms. As a result, in Fig. 21 we captured the results of the protected streams #1 and #2. The results are exponentially better with TAPRIO than without it, as can be observed by comparing Figs. 19 to 21.

A remarkable finding was that stream #2 yielded better results than stream #1 in the latencies of the TX_SOFTWARE timestamp. Fig. 22 shows the results of only stream #2 in contraposition to the aggregate of both streams shown in Fig. 21. Even though stream #1's TX_SOFTWARE samples go up to nearly 200 ms, the worst TX_SOFTWARE samples stream #2 do not reach 50 ms. This unveils that stream #1 is effectively acting as a guard band of stream #2 in regards to Qdisc processing delays.

In addition to the above, we also experimented with other parameters that could potentially affect the latencies across the transmit path. We observed that neither pinning the NIC Tx/Rx queues used by each stream to an exclusive CPU, disabling GSO and TSO, raising the NIC's ring buffer size from 256 to 4096, nor using a pacing timer in the iperf3 streams made any difference. The measured latencies were similar and the outcome of the full-bandwidth TCP test did not change. Hence, results of these configurations are not graphically presented for brevity.

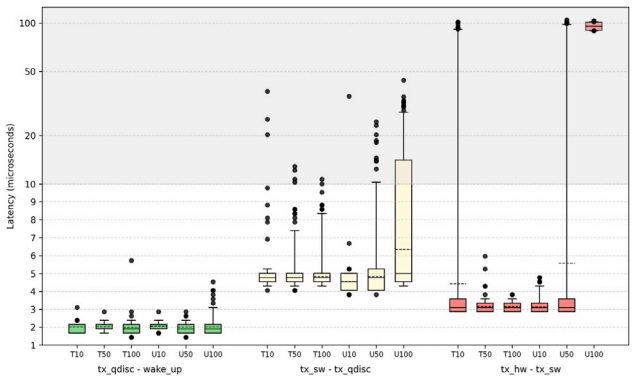


Fig. 22. Transmit path latencies with server under iperf3 load using real-time scheduling, CPU isolation, and the TAPRIO Qdisc (Stream #2).

5.4. Discussion and assessment of results

Over this section, we have performed a granular and comprehensive benchmarking of the network transmit and receive path performance applied to a cyclic control loop with stringent low-latency and determinism requirements. There are valuable conclusions to take from the experimental evaluation carried out in this section. Real-time scheduling and CPU isolation and affinity are both very important to guarantee the timeliness of latency-bound applications. The CPU affinity of network hardware IRQs did have an effect, but it was minimal compared to the real-time scheduling and CPU affinity of the applications that send and receive the real-time streams. On the other hand, we also identified other mechanisms and tuning parameters that did not seem to have a meaningful impact on specifically improving low-latency, deterministic communication, such as segmentation and receive offload capabilities (GRO, TSO, and GSO), some NAPI parameters, or the NIC's ring buffer size.

As far as the TAPRIO Qdisc evaluation in the transmit path is concerned, it was found (Figs. 21 and 22) that real-time traffic classes preceded by another real-time traffic class that involved very low throughput yielded better latency bounds than those preceded by a best-effort traffic class where high-throughput TCP or UDP streams were being transmitted. This behavior is likely due to the fact that the TAS is enforced by the Qdisc at the Traffic Control layer, but packets may still be traversing subsequent points of the transmit path in the kernel after the TAPRIO Qdisc has closed the gate of the best-effort stream. Hence, a preceding real-time traffic class slots effectively acts as a guard band shields the subsequent real-time slot from the final packets of the previous best-effort traffic class.

Overall, it has to be considered the uncertainty of some of the results, a critical aspect considering that deterministic communications require consistent bounded latency in order to meet their intended deadline each period, regardless of whether the average latency is low. Two things can be highlighted in this regard. On the one hand, often, if was found that the results with the server under emulated load were highly dependent on the profile used to emulate background load in a mixed-criticality system. This emphasizes that it is highly important to understand and profile the environment in which the control-loop applications are running, as it can determine whether the capabilities that available purely in software in Linux (preemptable kernel, real-time scheduling policies, fine-grained CPU configurations, TAPRIO Qdisc, etc.) are satisfactory enough to provide the guarantees that these applications need. On the other hand, incorporating hardware with TSN offloading capabilities, or even dedicated TSN NICs, would likely yield improvements, particularly in the latencies incurred in the paths closest to the hardware. For example, NICs like the Intel® I210 and Intel® I225 support at least partial offload of the TAPRIO Qdisc to the

hardware. Even if these NICs are not typical of general-purpose edge-computing or cloud platforms due to throughput limitations or lack of virtualization support (e.g. SR-IOV), a server could be equipped with two types of NICs towards the next hop of the network. This way, even if the network itself is converged and shared thanks to TSN, the network processing in the end station would be better isolated from best effort traffic.

The measurements carried out in this section are deemed to provide useful quantitative guidelines concerning the impact of several mechanisms to shield real-time applications and traffic against system noise and best-effort traffic. Yet, it is also important to take into account when it comes to designing the optimal system configurations that these mechanisms are not unlimited resources; the number of CPUs is finite, and the execution of applications consumes time, both in terms of CPU –the scheduling and processing of the applications–, and of TAS slots –the real-time traffic exchanged by the applications. Furthermore, it was found that leaving short time slots for best-effort traffic can starve said traffic, at least when TAS is implemented in software via the TAPRIO Qdisc without NIC offloading support. Therefore, these mechanisms shall be configured appropriately according to aggregate requirements and available resources, considering that applications will have varying levels of deadline or latency constraints, down to best-effort applications. For applications that do not have a very stringent time requirement, using a real-time scheduling priority without an exclusive CPU may suffice, although, in this case, the existence of significant network stress was found to degrade network stack latency in particular (for example seen in Fig. 18). An important consideration is, however, that if the system is contended enough that periods of several real-time applications overlap in time, especially within the same exclusive CPU or in non-isolated CPUs, their SCHED_FIFO or SCHED_RR priorities would have to be sorted according to the time criticality of each application. In this case, using the SCHED_DEADLINE policy with a carefully planned schedule may be more suitable to contextually prioritize tasks according to their deadline.

6. Conclusion

In this paper we have thoroughly evaluated the impact of real-time and TSN mechanisms and configurations in Linux systems for supporting deterministic containerized applications. For this purpose, an industrial control loop that consists on a cyclic layer-2 communication between a vPLC and field devices was emulated and put under test. The capabilities we have examined include scheduling policies and priorities, CPU isolation and affinity, IRQ affinity, and the TAS using the TAPRIO Qdisc. For each test, several timestamps have been collected in order to granularly evaluate latencies in the transmit and/or receive paths of the real-time streams. We have also studied the performance under conditions where the real-time workload contended with best-effort background traffic, so as to emulate the mixed-criticality applications present in edge clouds. We believe that our study yields valuable insights to assess the viability of using general-purpose Linux servers with off-the-shelf software to support deterministic containerized applications, and to comprehend how to optimally configure them for this purpose.

CRediT authorship contribution statement

Jorge Sasiain: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Jasone Astorga:** Writing – review & editing, Supervision, Project administration, Funding acquisition. **Mariví Higuero:** Writing – review & editing, Supervision, Project administration, Funding acquisition. **David Franco:** Writing – review & editing, Visualization. **Asier Atutxa:** Writing – review & editing, Visualization. **Eduardo Jacob:** Writing – review & editing, Supervision, Project administration, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported in part by the National Project funded by MICIU/AEI/ 10.13039/501100011033 titled “Enabling Native-AI Secure Deterministic 6G Networks for Hyper-Connected Environments” (6G-INSPIRE) under Grant PID2022-137329OB-C44, and in part by the Smart Networks and Services Joint Undertaking (SNS JU) under Agreement 101096456 (NANCY).

Data availability

Data will be made available on request.

References

[1] X. Xu, Y. Lu, B. Vogel-Heuser, L. Wang, Industry 4.0 and industry 5.0—Inception, conception and perception, J. Manuf. Syst. 61 (2021) 530–535.
[2] S. Vitturi, C. Zunino, T. Sauter, Industrial communication systems and their future challenges: Next-generation ethernet, IIoT, and 5G, Proc. IEEE 107 (6) (2019) 944–961.
[3] D.G. Pivoto, L.F. De Almeida, R. da Rosa Righi, J.J. Rodrigues, A.B. Lugli, A.M. Alberti, Cyber-physical systems architectures for industrial internet of things applications in industry 4.0: A literature review, J. Manuf. Syst. 58 (2021) 176–192.
[4] M. Gaffurini, P. Bellagente, A. Depari, A. Flammini, E. Sisinni, P. Ferrari, Virtual plc in industrial edge platform: Performance evaluation of supervision and control communication, IEEE Trans. Instrum. Meas. (2024).
[5] R. Queiroz, T. Cruz, J. Mendes, P. Sousa, P. Simões, Container-based virtualization for real-time industrial systems—A systematic review, ACM Comput. Surv. 56 (3) (2023) 1–38.
[6] M. Sollfrank, F. Loch, B. Vogel-Heuser, Exploring docker containers for time-sensitive applications in networked control systems, in: 2019 IEEE 17th International Conference on Industrial Informatics, INDIN, Vol. 1, IEEE, 2019, pp. 1760–1765.
[7] F. Hofer, M.A. Sehr, A. Iannopollo, I. Ugalde, A. Sangiovanni-Vincentelli, B. Russo, Industrial control via application containers: Migrating from bare-metal to IAAS, 2019, arXiv preprint arXiv:1908.04465.
[8] V. Struhár, S.S. Craciunas, M. Ashjaei, M. Behnam, A.V. Papadopoulos, React: Enabling real-time container orchestration, in: 2021 26th IEEE International Conference on Emerging Technologies and Factory Automation, ETFA, IEEE, 2021, pp. 1–8.
[9] L. Abeni, Virtualized real-time workloads in containers and virtual machines, J. Syst. Archit. 154 (2024) 103238.
[10] T. Cucinotta, L. Abeni, M. Marinoni, A. Balsini, C. Vitucci, Reducing temporal interference in private clouds through real-time containers, in: 2019 IEEE International Conference on Edge Computing, EDGE, IEEE, 2019, pp. 124–131.
[11] S. Fiori, L. Abeni, T. Cucinotta, Rt-kubernetes: containerized real-time cloud computing, in: Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing, 2022, pp. 36–39.
[12] M. Barletta, M. Cinque, L. De Simone, R. Della Corte, Achieving isolation in mixed-criticality industrial edge systems with real-time containers, in: 34th Euromicro Conference on Real-Time Systems, ECRTS 2022, Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2022, pp. 15:1–15:23.
[13] P. Karachatzis, J. Ruh, S.S. Craciunas, An evaluation of time-triggered scheduling in the linux kernel, in: Proceedings of the 31st International Conference on Real-Time Networks and Systems, 2023, pp. 119–131.
[14] G. Albanese, R. Birke, G. Giannopoulou, S. Schönborn, T. Sivanthi, Evaluation of networking options for containerized deployment of real-time applications, in: 2021 26th IEEE International Conference on Emerging Technologies and Factory Automation, ETFA, IEEE, 2021, pp. 1–8.
[15] M. Maliosz, I. Moldován, M. Máté, C. Simon, J. Harmatos, Deterministic local cloud for industrial applications, in: 2023 IEEE 19th International Conference on Factory Communication Systems, WFCS, IEEE, 2023, pp. 1–8.
[16] A. Garbugli, L. Rosa, A. Bujari, L. Foschini, Kubernetsn: A deterministic overlay network for time-sensitive containerized environments, in: ICC 2023-IEEE International Conference on Communications, IEEE, 2023, pp. 1494–1499.
[17] D. Balla, I. Moldován, M. Máté, M. Maliosz, J. Harmatos, Time sensitive industrial applications in kubernetes, in: NOMS 2024-2024 IEEE Network Operations and Management Symposium, IEEE, 2024, pp. 1–7.

[18] Y. Wang, S. Yang, X. Ren, P. Zhao, C. Zhao, X. Yang, IndustEdge: A time-sensitive networking enabled edge-cloud collaborative intelligent platform for smart industry, IEEE Trans. Ind. Inform. 18 (4) (2021) 2386–2398.
[19] M. Szalay, P. Matray, L. Toka, Real-time faas: Towards a latency bounded serverless cloud, IEEE Trans. Cloud Comput. 11 (2) (2022) 1636–1650.
[20] H. Chahed, M. Usman, A. Chatterjee, F. Bayram, R. Chaudhary, A. Brunstrom, J. Taheri, B.S. Ahmed, A. Kassler, AIDA—A holistic AI-driven networking and processing framework for industrial IoT applications, Internet Things 22 (2023) 100805.
[21] M. Eppler, J. Schenk, T. Gruner, A. Zirkler, H. Mueller, A. Blenk, FabOS: Hooking up container platforms with time-sensitive networks, in: Proceedings of the 1st Workshop on Enhanced Network Techniques and Technologies for the Industrial IoT to Cloud Continuum, 2023, pp. 29–34.
[22] R. Andreoli, R. Mini, P. Skarin, H. Gustafsson, J. Harmatos, L. Abeni, T. Cucinotta, A multi-domain survey on time-criticality in cloud computing, IEEE Trans. Serv. Comput. (2025).
[23] IEC/IEEE, IEC/IEEE 60802 TSN Profile for Industrial Automation, URL https://1.ieee802.org/tsn/iec-ieee-60802/.
[24] 5G Alliance for Connected Industries and Automation (5G-ACIA), Integration of 5G with time-sensitive networking for industrial communications, 2021.
[25] 3GPP, ETSI TR 22 832 V17.4.0; 3rd generation partnership project; technical specification group services and system aspects; study on enhancements for cyber-physical control applications in vertical domains; stage 1 (release 17), 2021.
[26] IEEE Standard for Local and Metropolitan Area Networks–Bridges and Bridged Networks, IEEE Std 802.1Q-2022 (Revision IEEE Std 802.1Q-2018), 2022, pp. 1–2163, http://dx.doi.org/10.1109/IEEESTD.2022.10004498.
[27] IEEE Standard for Local and Metropolitan Area Networks – Bridges and Bridged Networks - Amendment 25: Enhancements for Scheduled Traffic, IEEE Std 802.1Qbv-2015 (Amendment to IEEE Std 802.1Q-2014 as amended by IEEE Std 802.1Qca-2015, IEEE Std 802.1Qcd-2015, and IEEE Std 802.1Q-2014/Cor 1-2015), 2016, pp. 1–57, http://dx.doi.org/10.1109/IEEESTD.2016.8613095.
[28] IEEE Standard for Local and Metropolitan Area Networks–Timing and Synchronization for Time-Sensitive Applications, IEEE Std 802.1AS-2020 (Revision IEEE Std 802.1AS-2011), 2020, pp. 1–421, http://dx.doi.org/10.1109/IEEESTD.2020.9121845.
[29] IEEE Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems, IEEE Std 1588-2019 (Revision OfIEEE Std 1588-2008), 2020, pp. 1–499, http://dx.doi.org/10.1109/IEEESTD.2020.9120376.
[30] TSN Documentation Project for Linux*, URL https://tsn.readthedocs.io/.



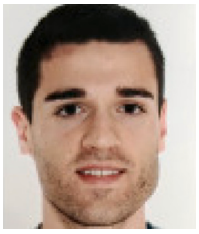
Jorge Sasiain received his MSc. degree in Telecommunication Engineering from the University of the Basque Country (UPV/EHU) in 2020. He joined the Communications Engineering Department of the UPV/EHU as a researcher in the I2T (Engineering and Research on Telematics) research lab in 2018. His research interests include network functions virtualization (NFV), software-defined networking (SDN), time-sensitive networking (MEC), and their applications in 5G networks and industrial environments. Currently, he is a Ph.D. student in Mobile Network Information and Communication Technologies at the UPV/EHU.



Jasone Astorga received the B.Sc. and M.Sc. degrees in telecommunication engineering and the Ph.D. degree from the University of the Basque Country (UPV/EHU), in 2004 and 2013, respectively. From 2004 to 2007, she worked with Nextel S.A., a Telecommunications Enterprise. She joined the UPV/EHU, as a Lecturer, in 2007, and as a Researcher with the I2T Research Laboratory. She is currently an Assistant Professor with the UPV/EHU. She has participated in several research projects at the local, national, and European levels. She has supervised several Ph.D. theses. Her research interests include software defined networking and network function virtualization for industrial communications, IP-enabled wireless sensor networks, and cybersecurity.



Marivi Higuero received the B.S. and M.S. degrees in electrical engineering, and the Ph.D. degree from the University of the Basque Country (UPV/EHU), in 1992 and 2005, respectively. She was with Sarenet, an Internet Service Provider, as a member of the technical department. She is currently an Assistant Professor with the Communications Engineering Department, UPV/EHU, where she is also a member of the I2T Research Laboratory. Her research interests include computer networks and services, sensor environments, mobility, and security.



David Franco received the B.Sc. and M.Sc. degrees in Telecommunication Engineering from the University of the Basque Country (UPV/EHU) in 2016 and 2018, respectively. He also received the Ph.D. in Mobile Network Information and Communication Technologies from the UPV/EHU in 2025. He joined the UPV/EHU in 2016 as a researcher in the I2T (Engineering and Research on Telematics) research group, and he has been a Ph.D. Assistant Professor in the Communications Engineering Department of UPV/EHU since 2025. His research focuses on softwarized and programmable networks, and network virtualisation, applied to traffic engineering and secure communication systems.



Asier Atutxa received his B.Sc. and M.Sc. in Telecommunications Engineering in 2018 and 2020 respectively. In 2025, he presented his Ph.D. thesis “Efficient Security for 5G/6G Networks: Performance Enhancements for Authentication and Confidentiality in Core and Radio Access Networks” in the inter-university Doctoral Program in Information and Communications Technologies in Mobile Networks. In 2018, he joined the I2T (Telematics Research and Engineering) research group, first as a student in educational cooperation and later as a contracted researcher. Since September 2025, he has been an Assistant Professor at the University of the Basque Country (EHU). His research interests include cybersecurity, quantum communications, and heterogeneous networks in industrial environments.



Eduardo Jacob received the B.Sc. degree in industrial engineering and the M.Sc. degree in industrial communications and electronics in 1987 and 1991, respectively, and the Ph.D. degree in communications engineering from the University of the Basque Country (UPV/EHU) in 2001. He worked two years in a public research and development telecommunications enterprise. He had a management role for several years as IT Director in the private sector. Since 1994 he started to work full time at the University of the Basque Country, where he was the first Head of the Department of Communications Engineering from 2012 to 2016. He is currently a Full Professor at the same university. He also leads the I2T Research Laboratory. He also directed several Ph.D. theses and managed several research projects at local, national, and European levels. His research interests include applying software-defined networks to industrial communications, cybersecurity in distributed systems, software-defined wireless sensor networks, and in-network processing.